**Assignment 4 Report**

**2019320003 박준**

**Code Description & Implementation Details**

아래는 main.cpp에 대한 설명 및 구현 원리이다. 문단의 앞에 위치한 번호는 해당 파일의 몇 번째 줄에 위치한 코드인지를 나타낸다. 주석 및 처음부터 있던 스켈레톤 코드의 설명은 꼭 설명이 필요한 경우가 아니면 생략하였다.

**8~16:** include한 헤더 파일들. ("Angel.h" 추가)

**23:** pi, 즉 원주율을 상수화했다. 후술할 함수들에서 각도 계산 시 degree와 radian을 변환할 때 쓰인다.

**25~30:** skeleton code

**32~33:** 3가지의 DVR 모드를 결정하는 변수. keyboard 함수에서 1, 2, 3에 따라 각기 다른 DVR 모드를 사용할 수 있다.

**35~38:** 셰이더로 전달할 모델 행렬, 뷰 행렬, 투상 행렬

**43~72:** 주어진 3D 데이터를 화면에 표시하기 위한 Bounding Box의 vertex/index 정보. 이들은 후술할 VAO, VBO, EBO로 전달된다.

**74~75:** Bounding Box의 VAO, VBO, EBO. 이들은 후술할 init() 함수에서 생성된다.

**77~79:** 창 너비/높이. 프로그램에서 창 크기에 대한 정보가 필요할 때 쓰이는 변수이다.

**81~84:** 카메라의 eye, at, up 변수.

**86~89:** 트랙볼 관련 변수. isTrackballRotating은 현재 사용자가 트랙볼을 회전시키고 있는지 여부를 나타낸다. tb_startPoint는 사용자가 트랙볼을 잡았을 때 마우스의 위치를 트랙볼로 투영한 위치를 나타낸다. trackballRotate는 트랙볼을 마우스가 잡아 이끄는 방향으로 회전시키는 행렬이다.

**91~94:** Zoom 관련 변수. zoom 변수를 조작해서 Perspective 함수의 fovy로 전달하면 zoom 기능을 구현할 수 있다. isZooming은 현재 사용자가 우클릭으로 zoom 기능을 사용하고 있는지 여부를 나타낸다. prevY는 마우스의 바로 전 Y좌표를 나타내며, isZooming이 true일 때 prevY의 값과 현재의 Y좌표값을 비교한 후 좌표에 차이가 있으면 사용자가 우클릭 후 마우스를 올리거나 내렸다는 뜻이므로 zoom 값에 수정을 가한다.

**96~97:** transfer function의 현재 색 구성을 실시간으로 셰이더로 전달하기 위한 변수.

**101~108: getRotation -** 주어진 4x4 행렬에서, 좌상단의 3x3 행렬 부분을 제외하고 전부 0으로 초기화하되, 4행 4열만 1로 둔다. 행렬의 '회전' 성분만 얻을 때 쓰인다.

**110~132: rotate –** 회전축과 회전각도를 주면, 그에 따른 회전 행렬을 반환한다.

**134~154: projectMouseToSphere -** 화면 위 마우스의 좌표가 주어지면 트랙볼 표면으로 투영하여 3D 좌표로 변환한다. 이때, 트랙볼의 반지름을 나타내는 변수 r은 화면의 너비/높이 중 더 짧은 부분의 절반과 같다. 마우스의 좌표와 트랙볼의 중심과의 거리가 반지름보다 먼 경우에는 가장 가까운 트랙볼 표면의 좌표로 매핑한다.

**156~165: getTrackballRotation –** 트랙볼 표면의 두 좌표를 주면, 두 좌표 사이의 각을 구하여 트랙볼의 회전 행렬을 반환한다. 원리는 lecture note에 있는 원리와 유사하다.

**214~215:** 창의 크기가 변경되는 경우, 상술한 창 너비/높이 변수에 현재 창 너비/높이 값을 저장한다.

**221~224:** DVR 모드 전환. 키보드 입력 1, 2, 3에 따라 각기 다른 DVR 모드를 사용할 수 있다.

**238:** DVR 모드를 결정하는 mode 변수를 셰이더로 전달한다.

**240~241:** transfer function의 현재 색 구성을 실시간으로 텍스처 형태로 저장한다.

**243~249:** 기존의 eye와 up에 트랙볼의 회전을 적용한 newEye와 newUp을 계산한다. 트랙볼의 회전을 카메라에 적용시키는 이유는, 사용자의 눈에는 자신이 트랙볼을 잡고 움직이는 것처럼 보이지만, 이 프로그램은 실제로는 트랙볼을 회전하면 Bounding Box는 카메라의 at 위치에 가만히 있고 카메라가 트랙볼 회전 방향의 반대 방향으로 움직이는 원리이기 때문이다. 이때 카메라 회전 시 at을 중심으로 공전시켜야 하므로, newEye와 newUp 계산 시 getRotation 함수를 이용하여 뷰 행렬의 회전 상태를 이용해야 한다. 이렇게 트랙볼을 회전시켜 얻은 새 카메라 좌표는 나중에 frag 셰이더의 eyePosition 변수로 전달된다.

**254~255:** 투상 행렬을 구한다. 이때 fovy 매개변수에 90 * zoom이 전달된다. Zoom은 1로 초기화되므로, fovy의 초기값은 90이다.

**257~261:** 셰이더에 모델 행렬, 뷰 행렬, 투상 행렬, eyePosition의 값을 전달한다.

**263~265:** VAO를 이용하여 Bounding Box를 그린다.

**270: mouseClick –** 마우스 클릭 콜백 함수. 트랙볼 회전에 쓰이는 좌클릭과, 줌에 쓰이는 우클릭을 다룬다.

**271~284:** 좌클릭을 다룬다. 막 누른 경우 마우스의 트랙볼 투영 좌표와, 트랙볼이 현재 회전중이라는 상태를 저장한다. (꾹 누르는 경우는 후술할 mouseMotion 함수에서 다룬다.) 막 뗀 경우 카메라의 최종 위치를 계산하여 eye, up 그리고 뷰 행렬에 저장한 후, 트랙볼 회전 행렬을 초기화하고 트랙볼이 현재 회전중이 아니라는 상태를 저장한다.

**285~293:** 우클릭을 다룬다. 막 누른 경우 zoom 기능이 활성화되었다는 상태를 저장하고, 마우스의 y 위치에 따라 줌 기능을 조작하기 위해 prevY 값을 현재 마우스의 y값으로 초기화한다. (꾹 누르는 경우는 후술할 mouseMotion 함수에서 다룬다.) 막 뗀 경우 줌 기능이 비활성화되었다는 상태를 저장한다.

**296:** 마우스 움직임 콜백 함수.

**297~301:** 트랙볼이 회전 중일 때를 다룬다. 마우스의 현재 좌표를 트랙볼에 투영한 3D 좌표를 실시간으로 계산하고, 트랙볼 회전 행렬 변수인 trackballRotate의 값을 그에 따라 실시간으로 업데이트한다.

**302~310:** 줌 기능이 활성화되었을 때를 다룬다. 마우스를 우클릭한 상태로 위로 올리면 확대, 아래로 내리면 축소다. 단, zoom은 0.1~1.9 사이의 값만 가지도록 제한한다. prevY 값은 실시간으로 현재 Y값으로 반영한다.

**314~321:** glutPostRedisplay(); 부분을 idle 시 무조건 실행되도록 바꿨다.

**329~352:** VAO, VBO, EBO 생성

**364~369:** transfer function의 색 구성을 저장할 텍스처 생성

**371:** transfer function의 색 구성을 셰이더로 전달

다음은 volumeRendering.vert에 대한 설명이다.

**4~6:** 모델 행렬, 뷰 행렬, 투상 행렬을 main.cpp로부터 받아온다.

**11:** 정점의 월드 좌표를 pixelPosition으로 전달한다.

**12:** gl_Position에 정점의 절단 좌표를 전달한다.

다음은 volumeRendering.frag에 대한 설명이다.

**12:** DVR 모드를 main.cpp로부터 받아온다.

**14~26: intersectBox -** ray를 쏘았다고 가정하자. 함수에 ray의 출발 좌표(rayStart)와 ray

의 방향(rayDir)을 주고, Bounding Box의 위치와 크기 정보(즉 Bounding Box의 가장 작은 좌표와 가장 큰 좌표)를 주면, 그 ray가 Bounding Box를 뚫고 들어가는 곳과 나오는 곳을 매개변수 tMin과 tMax를 통해 전달하는 함수이다. 이렇게 얻은 tMin과 tMax는 나중에 카메라에서 나오는 ray가 Bounding Box를 통과하는지를 판단할 때 결정적인 역할을 한다. 구체적으로는 벡터 rayDir에 tMin ~ tMax 사이의 값을 곱한 후, 이를 점 rayStart에 더하면 그 결과로 점이 하나 나오는데(점+벡터=점) 해당 점은 Bounding Box 안에 있는 점이다.

**28~30: normalizePos –** 주어진 월드 좌표를 GLSL의 texture 함수의 2번째 매개변수인 vec3형 변수에 들어갈 값으로 정규화한다.

**38~39:** ray를 생성한다.

**41~45:** ray가 Bounding Box와 겹치는 부분을 구한다. 없으면 계산을 중단한다.

**47~49:** ray가 Bounding Box를 뚫고 들어가는 부분과 나오는 부분의 좌표를 구한다.

**51~52:** 샘플링 좌표와 좌표의 증분값을 설정한다. ray가 Bounding Box를 뚫고 들어가는 부분부터 시작하여, dt씩 증가시키며 샘플링하는 원리이다.

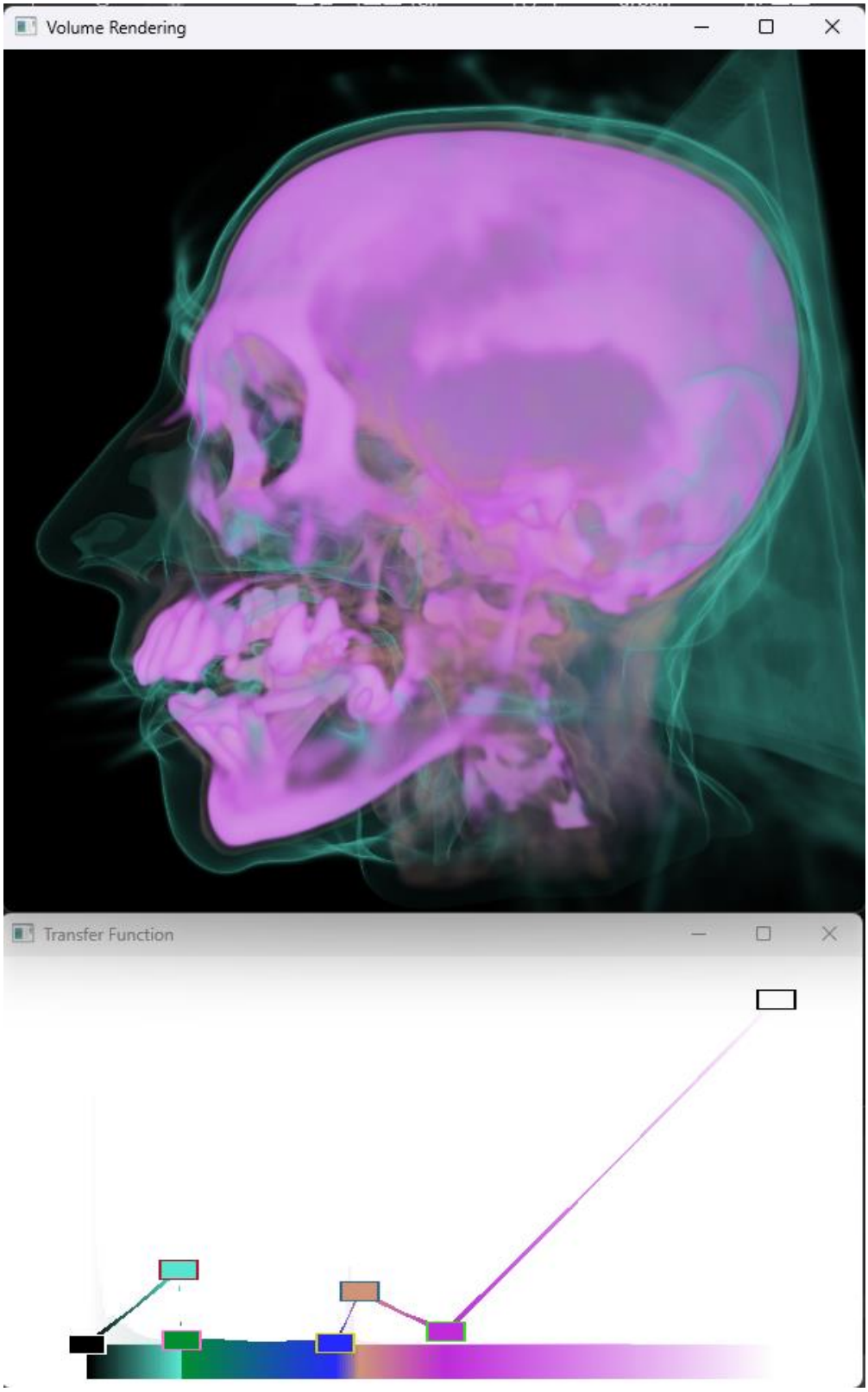
**54:** mode값에 따라 DVR 모드를 설정한다.

**57~73:** MIP 구현. 현재 ray의 샘플링 값들 중 최대값을 저장하여 gl_FragColor에 반영한다.

**75~116:** Isosurface Rendering 구현. Isovalue는 0.5로 설정하고, 샘플링을 하면서 ray를 따라 나아가다가 isovalue보다 큰 곳을 발견하면 Phong Shading을 구현했다. 원래 알고리즘대로라면 isovalue보다 큰 곳을 발견했을 때 반대 방향으로 움직이는 것을 반복하며 isovalue보다 큰 곳의 정확한 위치를 선별해야 하지만, 이 프로그램의 경우 현재 dt가 1이고 3D 데이터도 단위 길이가 1이므로 그럴 필요가 없기에 구현 시 CPU를 낭비하는 결과를 초래한다.

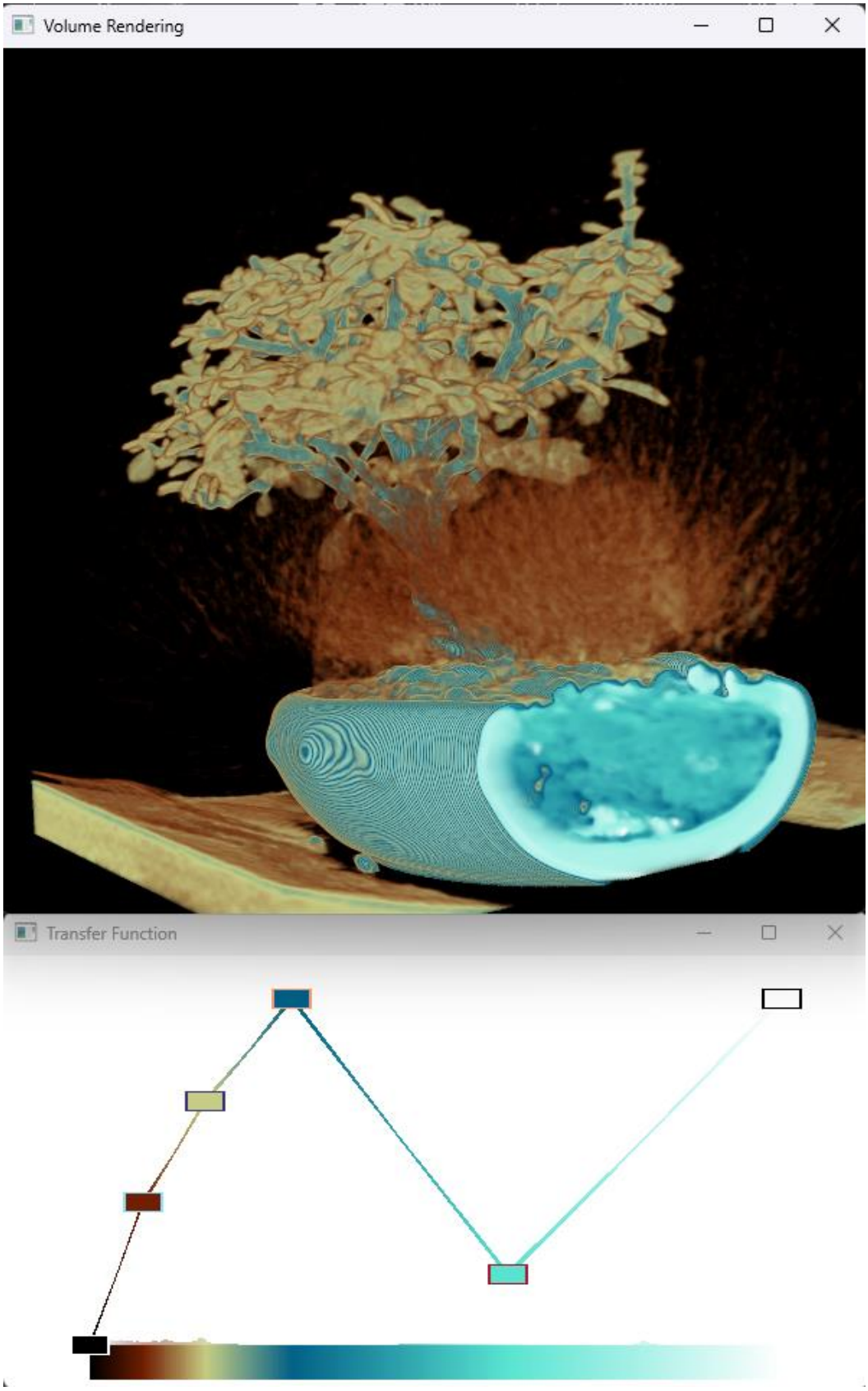
**118~139:** Alpha Compositing 구현. Lecture Note에 나온 점화식을 적용하되, alpha값을 적절히 조절했다. Ray를 쏘는 방향으로 샘플링을 하므로 Front-To-Back Compositing이며, 1이 넘으면 조기에 종료시킴으로써 Early Termination도 구현했다.

**Rendering Results**



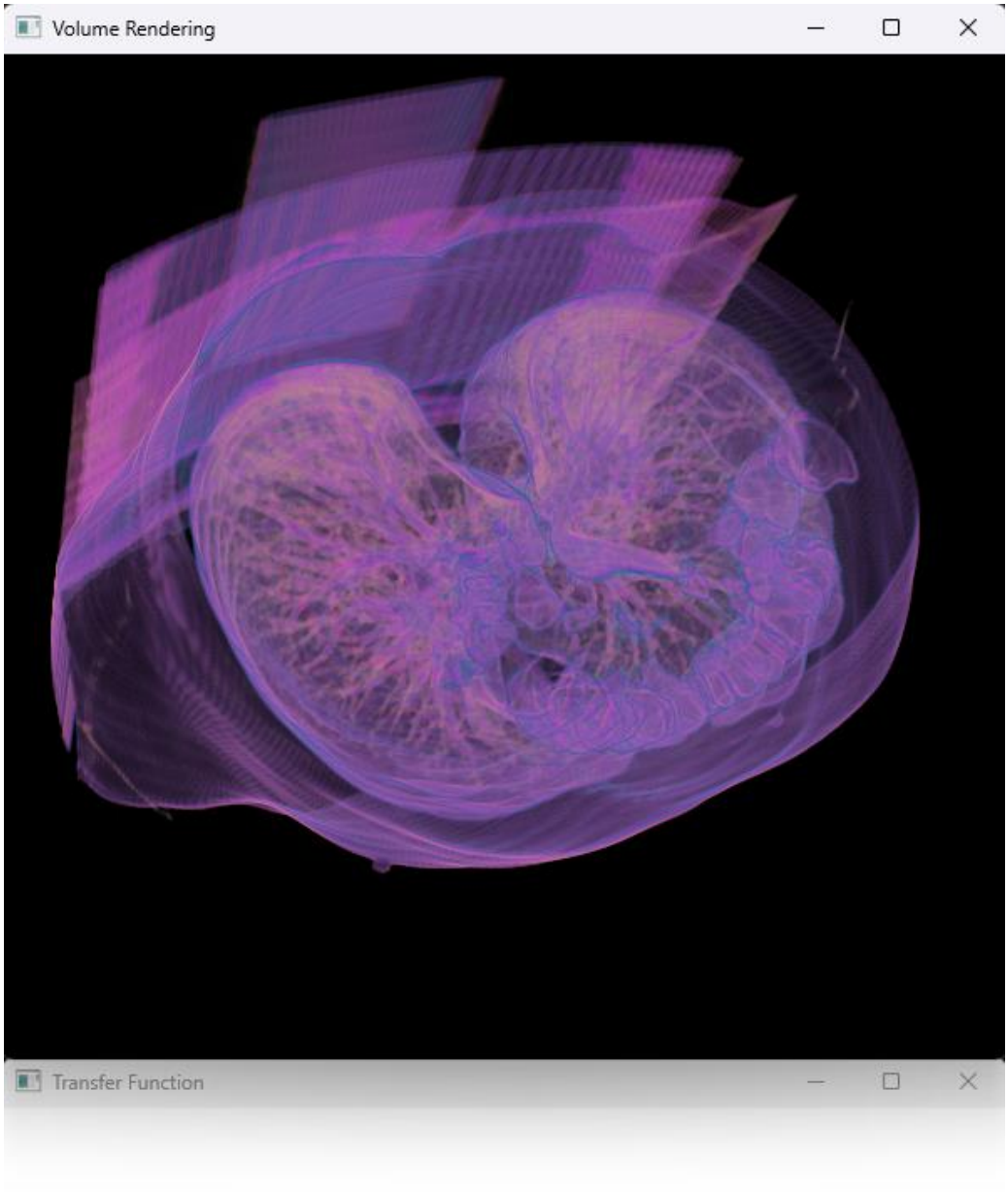

CThead_512_512_452.raw

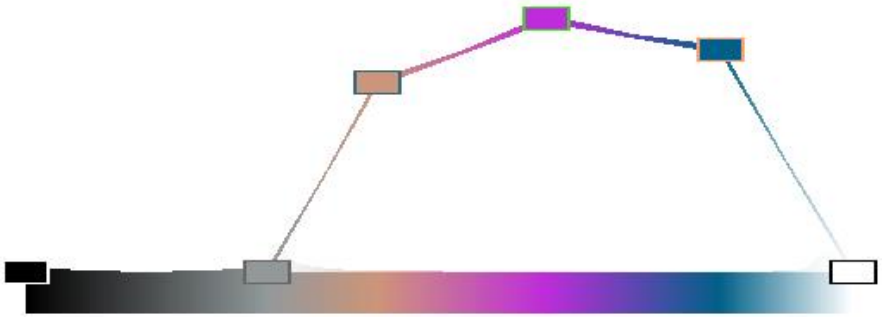
Dimension (x, y, z): 512, 512, 452




bonsai_256_256_256.raw

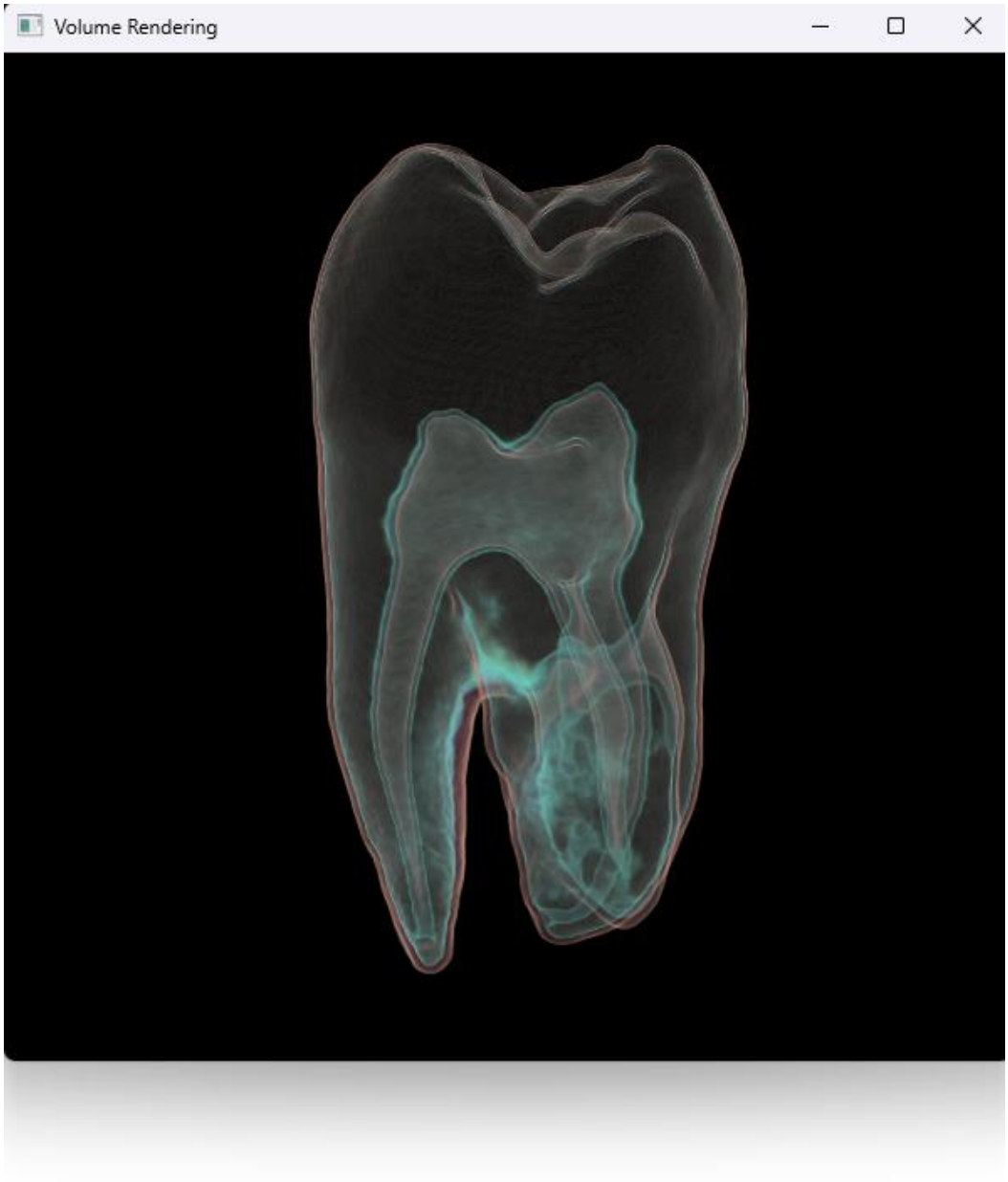
Dimension (x, y, z): 256, 256, 256

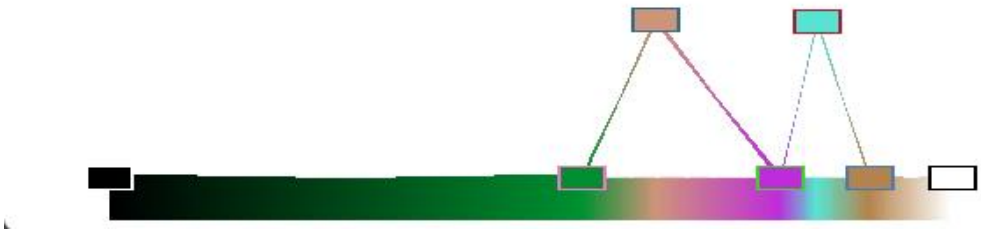


lung_256_256_128.raw

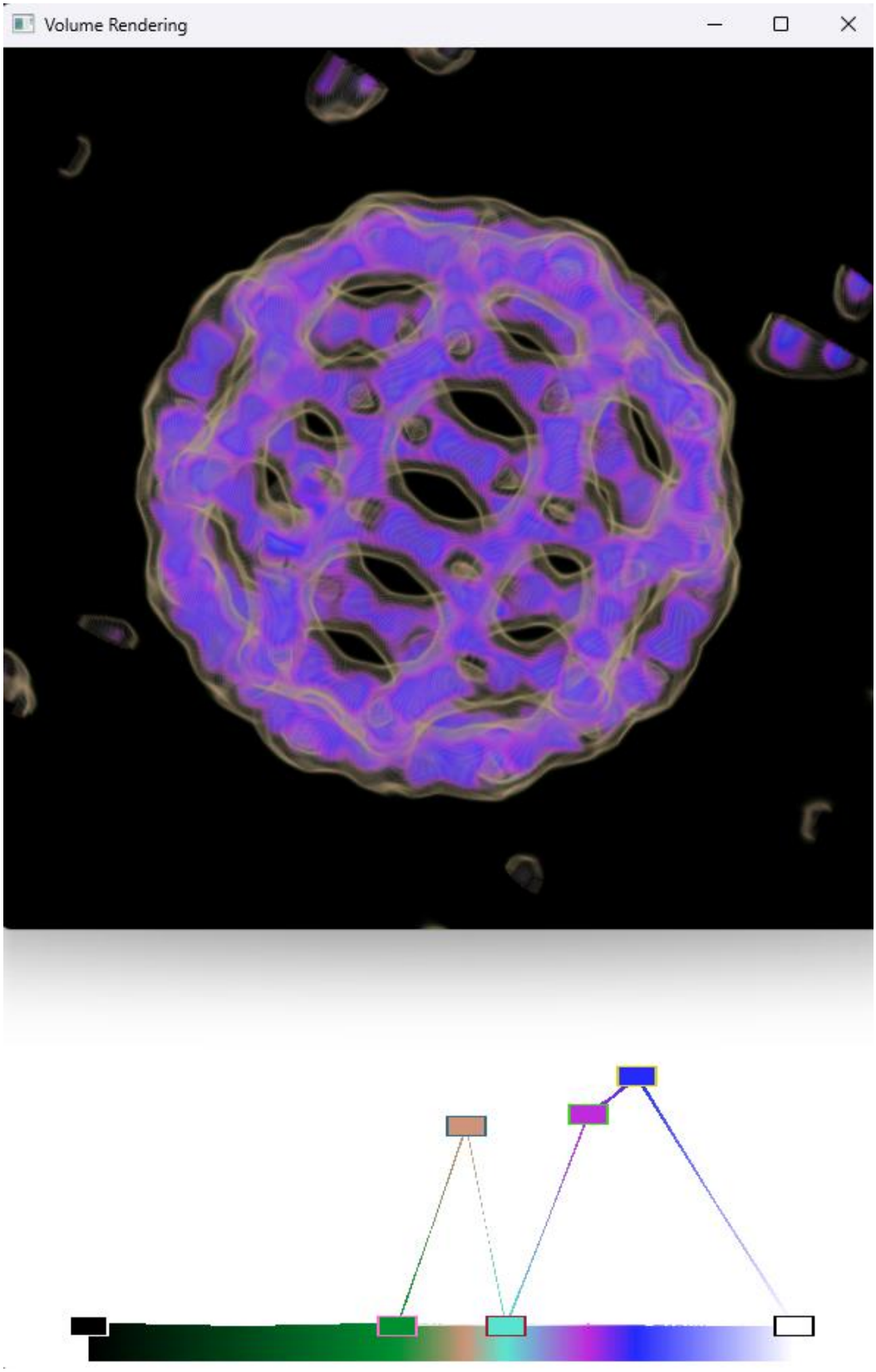
Dimension (x, y, z): 256, 256, 128



tooth_100_90_160.raw

Dimension (x, y, z): 100, 90, 160




Bucky_32_32_32.raw

Dimension (x, y, z): 32, 32, 32